Java Control Flow & Loops

Definitions • Java examples • output • real-world uses • flowcharts

1. What is control flow?

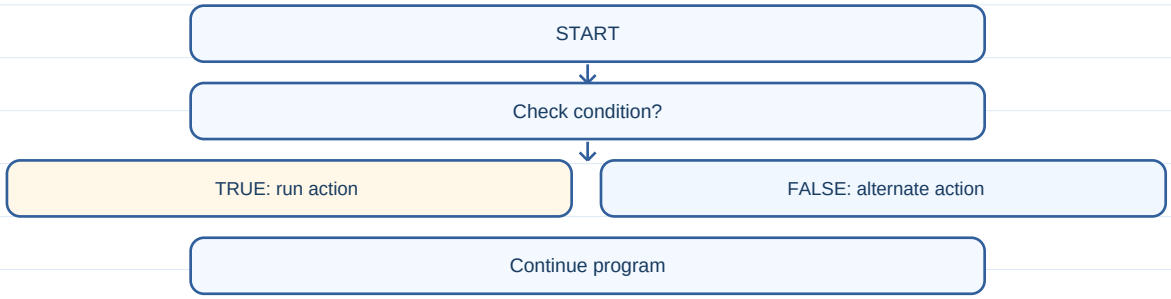
Definition: Control flow is the order in which Java statements execute. Normally, statements run from top to bottom, but decisions, loops and jump statements can change that order.

Why it matters: Programs must choose actions, repeat work, skip steps and finish at the correct time.

Structure	Purpose	Real-world analogy
Selection	Choose a path (if, switch)	Traffic signal chooses GO or STOP.
Iteration	Repeat work (loops)	Cashier scans every item in a basket.
Jump	Change normal flow	Exit a search early or skip a record.

2. Basic decision flow

A decision selects one of two paths



A condition is a boolean expression: it evaluates to true or false. Examples: `age >= 18` *and* `marks > 40`.

3. Small Java example

```
int temperature = 32;
if (temperature > 30) {
System.out.println("Hot day");
}
```

Output: Hot day

Real-world use: A weather app can show a heat warning when temperature crosses a threshold.

Remember: Use braces { } even for a one-line block. This reduces mistakes when code is extended.

if, if-else and nested if

Decision statements execute code according to boolean conditions.

1. if statement

Definition: An if statement runs its block only when its boolean condition is true.

```
if (condition) {
// runs only when condition is true
}

int stock = 5;
if (stock > 0) {
System.out.println("In stock");
}
```

Output: In stock

Real-world example: An online store shows "Add to cart" only when an item is available.

2. if-else statement

Definition: if-else chooses one of two blocks: if the condition is true, run if; otherwise, run else.

```
int marks = 72;
if (marks >= 40) {
System.out.println("Pass");
} else {
System.out.println("Try again");
}
```

Output: Pass

Real-world example: A result portal shows PASS or FAIL based on the passing score.

3. Nested if

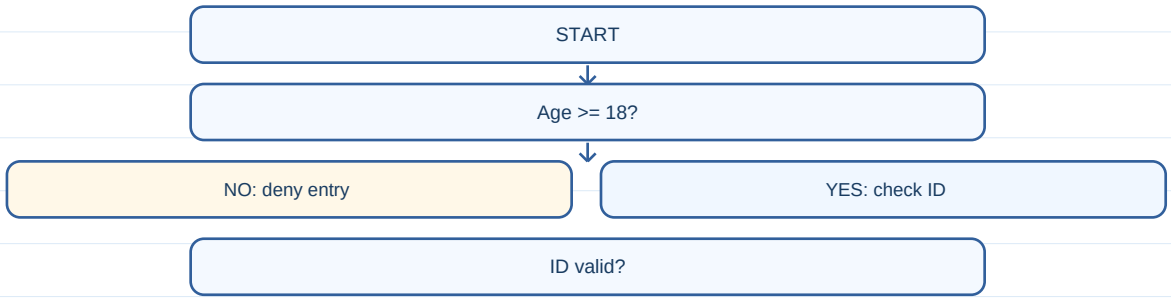
Definition: A nested if is an if statement inside another if or else block. The inner check runs only if program flow reaches it.

```
int age = 20;
boolean hasId = true;
if (age >= 18) {
if (hasId) {
System.out.println("Entry allowed");
}
}
```

Output: Entry allowed

Real-world example: Event entry checks age first, then checks whether the visitor has valid ID.

Nested decision: check age, then ID



Remember: Avoid too many nested levels. An else-if chain or an early return may be easier to read.

switch and fall-through

switch is useful when one expression is compared with several possible values.

1. switch statement

Definition: switch selects a matching case using an expression. Traditional switch supports types such as int, char, String and enum, plus certain related primitive wrapper types.

```
int day = 2;
switch (day) {
case 1: System.out.println("Monday"); break;
case 2: System.out.println("Tuesday"); break;
default: System.out.println("Other day");
}
```

Output: Tuesday

Real-world example: An ATM menu can select Withdraw, Deposit or Check balance.

2. Fall-through

Definition: In traditional colon-style switch, execution continues into the next case if break or another control transfer is missing. This is called fall-through.

```
int level = 2;
switch (level) {
case 1: System.out.println("Basic");
case 2: System.out.println("Standard");
case 3: System.out.println("Premium"); break;
default: System.out.println("Invalid");
}
```

Output: Standard, then Premium. Case 2 falls through into case 3.

Real-world example: Several membership levels may intentionally share features; fall-through can group their behavior.

3. Avoid accidental fall-through

```
switch (level) {
case 1: System.out.println("Basic"); break;
case 2: System.out.println("Standard"); break;
case 3: System.out.println("Premium"); break;
default: System.out.println("Invalid");
}
```

Modern arrow-style labels such as `case 1 -> ...` *do not fall through. Switch expressions can also produce a value.*

> *Remember: In a traditional switch, use break when you want to leave the switch. Document intentional fall-through.*

for, while and do-while loops

Loops repeat a block while the loop's condition or iteration rule allows it.

1. for loop

Definition: A for loop places initialization, condition and update in one header. It is often used when the number of repetitions is known.

```
for (int i = 1; i <= 3; i++) {
System.out.println(i);
}
```

Output: 1, 2, 3

Real-world example: Print the first 10 rows of a bill or process a fixed number of quiz questions.

2. while loop

Definition: A while loop checks its condition before each iteration. It can run zero times if the condition starts false.

```
int count = 1;
while (count <= 3) {
System.out.println(count);
count++;
}
```

Output: 1, 2, 3

Real-world example: Ask for a valid PIN while the entered PIN is incorrect and attempts remain.

3. do-while loop

Definition: A do-while loop checks its condition after executing the body, so the body always runs at least once.

```
int choice;
do {
choice = 0; // read menu choice here
System.out.println("Menu shown");
} while (choice != 0);
```

Output: Menu shown at least once.

Real-world example: Display an app menu once, then repeat until the user chooses Exit.

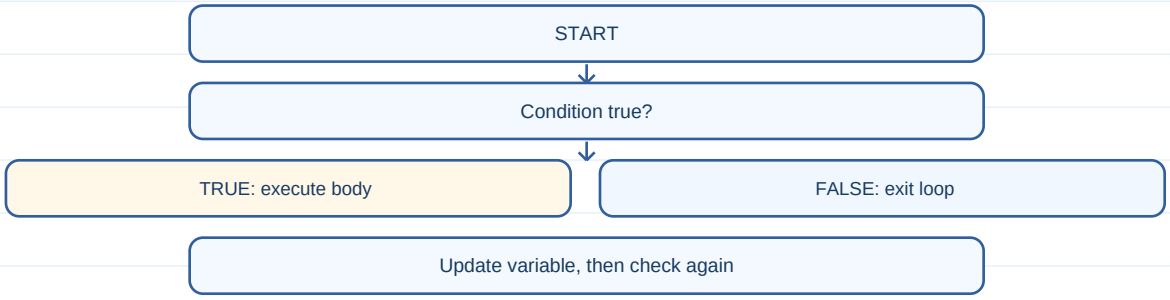
Loop	When condition checked	Minimum runs	Good fit
for	Before body	0	Known count / index
while	Before body	0	Repeat while true
do-while	After body	1	Must run once

Loop flowcharts and tracing

Follow the condition, body and update to understand each iteration.

1. while loop flowchart

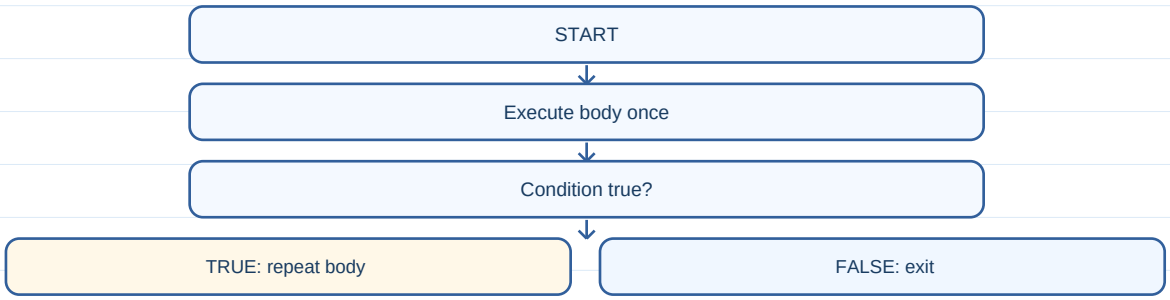
while checks before running the body



After the body and update, control returns to the condition. If it is false, the loop ends.

2. do-while flowchart

do-while runs the body before checking



3. Trace a for loop

```
int sum = 0;
for (int i = 1; i <= 4; i++) {
sum += i;
}
System.out.println(sum);
```

Pass	*i*	*sum after sum += i*
1	*1*	*1*
2	*2*	*3*
3	*3*	*6*
4	*4*	*10*

Output: 10

Real-world example: Add daily sales amounts to calculate a weekly total.

Remember: A loop needs progress toward stopping. If the condition never becomes false, an infinite loop may occur.

Enhanced for loop and nested loops

Use for-each to visit elements; use nested loops for repeated groups.

1. Enhanced for loop (for-each)

Definition: The enhanced for loop visits each element of an array or Iterable collection in sequence. It is convenient when the index is not needed.

```
int[] scores = {80, 90, 75};
for (int score : scores) {
System.out.println(score);
}
```

Output: 80, 90, 75

Real-world example: Display every product in a cart or total the prices in a list.

2. Standard for vs enhanced for

Standard for	Enhanced for
Provides an index such as i.	Provides the current element.
Can move backwards or by custom steps.	Visits elements in iteration order.
Useful when index is required.	Simple for straightforward traversal.

3. Nested loops

Definition: A nested loop is a loop inside another loop. For every outer-loop iteration, the inner loop completes its iterations unless a jump statement changes the flow.

```
for (int row = 1; row <= 3; row++) {
for (int col = 1; col <= 2; col++) {
System.out.print(row + "-" + col + " ");
}
System.out.println();
}
```

Output:
1-1 1-2
2-1 2-2
3-1 3-2

Real-world example: A timetable has rows for days and columns for periods. Nested loops can visit each day-period cell.

4. Pattern example

```
for (int r = 1; r <= 3; r++) {
for (int c = 1; c <= r; c++) {
System.out.print("* ");
}
System.out.println();
}
```

Output:
*
* *
* * *

Remember: For a grid with n rows and m columns, the inner body runs n × m times.

break and continue

These statements change the normal movement through a loop or switch.

1. break

Definition: An unlabeled break immediately exits the nearest enclosing loop or switch. Execution continues after that structure.

```
for (int n = 1; n <= 5; n++) {
if (n == 3) break;
System.out.println(n);
}
```

Output: 1, 2

Real-world example: Stop searching a customer list as soon as the required record is found.

2. continue

Definition: continue skips the rest of the current loop iteration and moves to the next iteration. In a for loop, the update expression runs before the next condition check.

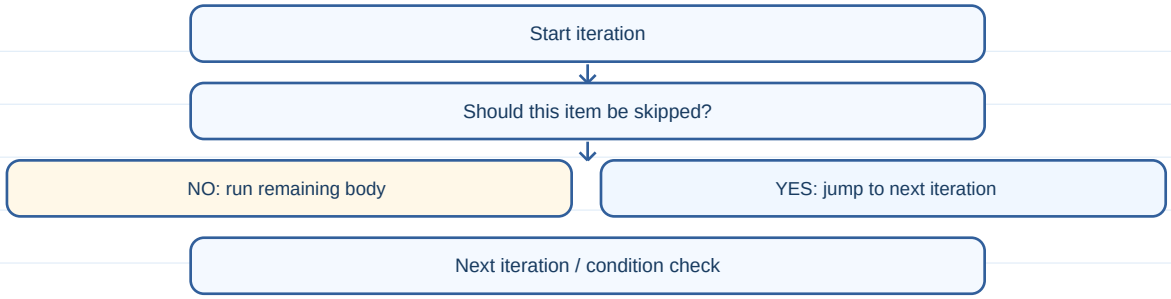
```
for (int n = 1; n <= 5; n++) {
if (n == 3) continue;
System.out.println(n);
}
```

Output: 1, 2, 4, 5

Real-world example: Skip cancelled orders while processing the remaining orders in a batch.

Statement	Effect	What happens next?
break	Leaves nearest loop or switch.	Continue after that structure.
continue	Skips current loop iteration.	Start the next iteration.

continue skips the remaining body



Remember: Unlabeled break exits only the nearest enclosing loop. Use a labeled break to exit an outer loop.

Labeled break and labeled continue

A label names an outer loop so a jump can target that loop.

1. Labeled break

Definition: A labeled break exits the statement associated with a specified label. It is useful for leaving multiple nested loops at once.

```
search:
for (int row = 0; row < 3; row++) {
for (int col = 0; col < 3; col++) {
if (row == 1 && col == 1) break search;
System.out.println(row + "," + col);
}
}
```

Output: (0,0), (0,1), (0,2), (1,0), then the labeled outer loop ends.

Real-world example: Stop scanning a seating chart as soon as a target seat is found.

2. Labeled continue

Definition: A labeled continue skips to the next iteration of the labeled loop. For an outer for loop, its update runs before the next condition check.

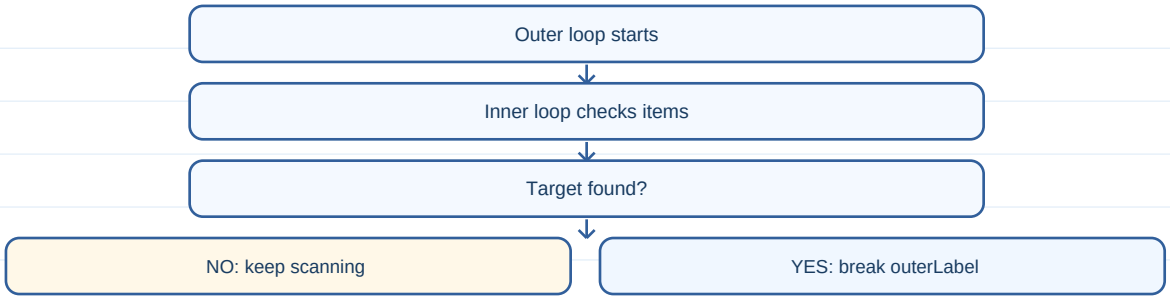
```
days:
for (int day = 1; day <= 3; day++) {
for (int slot = 1; slot <= 3; slot++) {
if (slot == 2) continue days;
System.out.println(day + "-" + slot);
}
}
```

Output: 1-1, 2-1, 3-1. Each continue jumps to the next day.

Real-world example: A schedule checker can skip the remaining slots for a day marked unavailable.

3. Flow idea

Labeled break leaves the named outer loop



Remember: Labels are identifiers followed by a colon, such as outer:. Use them sparingly; a helper method may be clearer.

return and reachability

return exits a method; reachability is checked by Java's compiler.

1. return statement

Definition: return terminates the current method. A void method can use `return;`*. A non-void method must provide a compatible value on every required path.*

```
static int square(int n) {
return n * n;
}
System.out.println(square(5));
```

Output: 25

```
static void printIfReady(boolean ready) {
if (!ready) return;
System.out.println("Ready!");
}
```

Real-world example: A validation method can return false immediately when input is invalid, avoiding unnecessary work.

2. Reachability

Definition: A statement is reachable when Java's compile-time rules allow execution to arrive at it. The compiler rejects certain unreachable statements.

```
int value = 10;
if (value > 0) {
System.out.println("Positive");
}
System.out.println("Done");
```

Both print statements are reachable. A statement placed immediately after an unconditional return in the same block is unreachable and causes a compile-time error.

```
static int getValue() {
return 10;
// System.out.println("Never reached"); // compile-time error
}
```

3. Do not confuse these statements

Concept	Meaning
break	*Exit nearest loop or switch.*
continue	*Skip to next iteration of a loop.*
return	*Exit the current method, optionally with a value.*
Reachability	*Whether Java's compile-time rules permit a statement to be reached.*

Remember: Reachability follows Java language rules, not simply a programmer's guess about what will happen at runtime.

Revision, mistakes and practice

Review the key rules and test yourself with short questions.

1. Quick revision

Topic	Remember
if / if-else	Condition must be boolean; select a path.
nested if	A decision inside another decision.
switch	Colon-style cases can fall through.
for / while	Condition checked before the body.
do-while	Body runs at least once.
enhanced for	Visit each array or Iterable element.
nested loops	Inner loop repeats for each outer iteration.
break / continue	Exit nearest structure / skip current iteration.
labeled jump	Target a named outer loop.
return	Exit the current method.
reachability	Compiler checks whether statements may be reached.

2. Common mistakes

- Writing `if (x = 5)` instead of a comparison such as `x == 5`.
- Forgetting to update a while-loop variable, causing an infinite loop.
- Forgetting break in a traditional switch and causing accidental fall-through.
- Assuming continue exits the loop — it skips only the current iteration.
- Omitting a return value on a required path in a non-void method.

3. Practice questions

Q1. Which loop always executes at least once? Answer: do-while.

Q2. What does an unlabeled break exit in nested loops? Answer: The nearest enclosing loop.

Q3. What is the output?

```
for (int i = 1; i <= 4; i++) {
if (i == 2) continue;
System.out.print(i + " ");
}
```

Answer: 1 3 4

Q4. Give a real-world use for nested loops. Answer: Processing rows and columns in a timetable.

4. Mini task

Write a menu program using do-while and switch. Show options 1) Add, 2) View, 0) Exit. Repeat until the user chooses 0. Use break in each traditional switch case.

Remember: Best habit: draw the flow first, trace variable values by hand, then write and test the Java code.

End of notes — Control Flow, Loops & Jump Statements